Retrieval-Augmented Fine-Tuning With Preference Optimization For Visual Program Generation

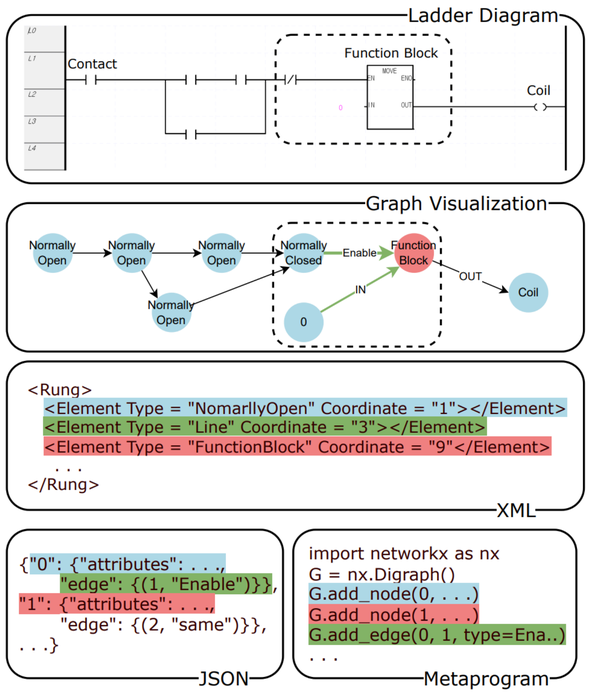
Deokhyung Kang[1*], Jeonghun Cho[1*], Yejin Jeon[1], Sunbin Jang[2], Minsub Lee[2], Jawoon Cho[3], Gary Geunbae Lee[1]

[1]Pohang University of Science and Technology, [2]Hyundai Mobis, [3]T&I Company (*: Equal Contribution)

Arxiv

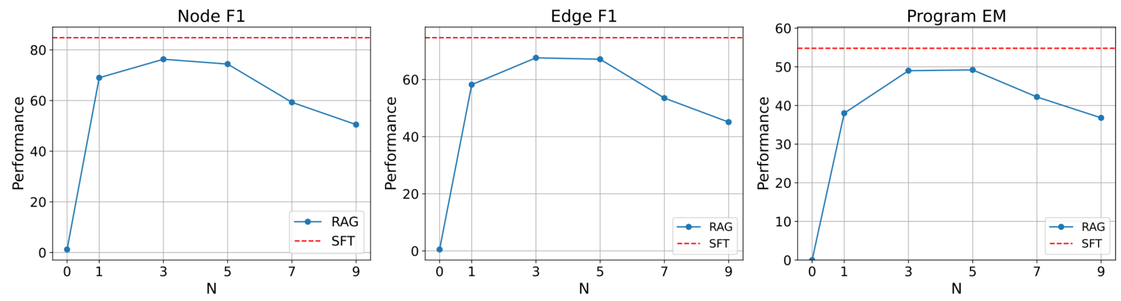
TL;DR: We propose the first approach that enables LLMs to generate Ladder Diagram (LD) programs–a widely used programming language in industrial automation-from user instructions.

Ladder Diagram



- **Ladder Diagram** is a **visual programming language (VPL)** used to control factory machinery via Programmable Logic Controllers (PLCs).
- It is **widely used for PLC** because its **circuit-like structure** makes it intuitive for engineers to design control logic.
- LLMs cannot directly generate visual elements. Thus, we convert LD into a **graph** and represent it in **three text formats**: XML, JSON, and Metaprogram.

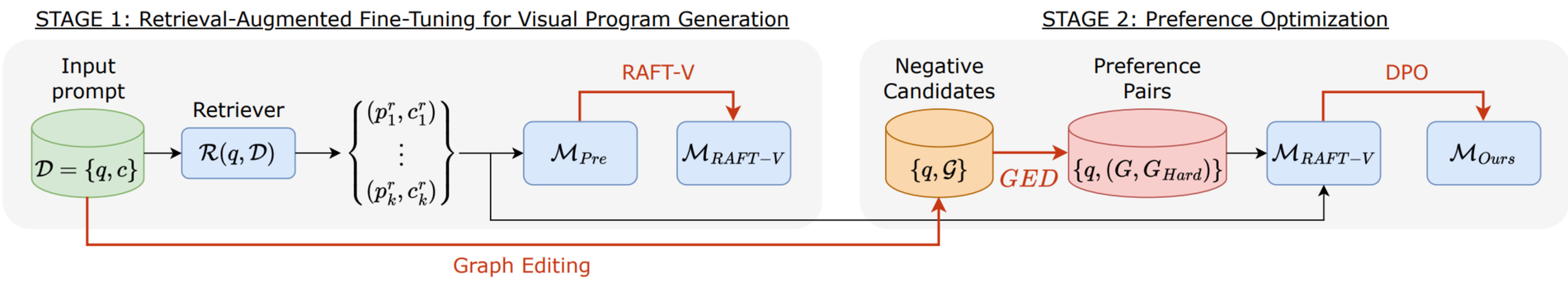
Challenge in generating LDs



SFT: Llama3.1-8B-Instruct, RAG: Llama3.1-70B-Instruct

- Most VPL generation studies rely on prompting-based methods.
- ⚠️ We find that **prompting-based methods struggle with LD generation.**
- **Why?** 🔍 LD follows **domain-specific configurations** (e.g. address mappings) that vary drastically across environments and are difficult to capture in a single prompt.

Our Solution: 2-stage training-based approach for LD generation



Input: User prompt $\boldsymbol{q}$ + Retrieved examples $[(\boldsymbol{p}_1^r, \boldsymbol{c}_1^r), .. (\boldsymbol{p}_k^r, \boldsymbol{c}_k^r)]$ -> Output: Generated LD $\boldsymbol{c}$

- Stage 1: Retrieval-Augmented Fine-Tuning (RAFT-V) : LD programs frequently reuse subroutines.
-> We retrieve similar prompt-code pairs from the dataset and fine-tune the model with them, both during training and inference.

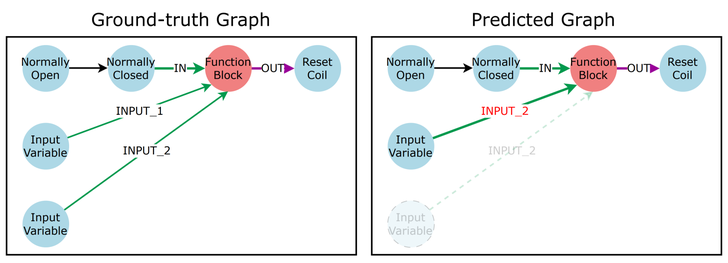
- Stage 2: Preference Optimization: LDs can be represented as a node-based graphs.
-> We generate preference pairs by applying graph edits to ground-truth LDs, then train the model to prefer correct LD using Direct Preference Optimization (DPO).

Algorithm 1 Graph Editing

Input: Random seed set S with $|S| = s$, Deletion ratio τ, Graph $G = (V, E)$ with $|V(G)| = n$ nodes and $|E(G)| = m$ edges
// Initialize the graph
$G' \leftarrow G, \mathcal{G} \leftarrow \emptyset$;
// Compute number of nodes to delete
$k \leftarrow \lfloor \tau \cdot |V(G')| \rfloor$;
for each i in S **do**
  if $k > 0$ **then**
    // Randomly select and reconnect nodes
    $\mathcal{X} \leftarrow$ random k nodes from G';
    Let $\mathcal{W} \rightarrow \mathcal{X} \rightarrow \mathcal{Y}$ be a path in G';
    Connect $\mathcal{W} \rightarrow \mathcal{Y}$, preserving the connectivity;
    // Remove selected nodes
    Remove node $\mathcal{X}$ from G';
  else
    // If no node is removed, copy a random node
    Copy a random node-edge pair $(v_{\text{copy}}, e_{\text{copy}}) \in G'$;
    Connect a randomly selected node v_{orig} in G' to v_{copy} using e_{copy};
  end if
  // Store the modified graph
  Add G' to set $\mathcal{G}$;
end for
Return $\mathcal{G} = \{G'_1, G'_2, \ldots, G'_s\}$

Experimental Results

Setup



- 14,124 LD programs from actual industrial environments, split into train/dev/test sets (13,124 / 500 / 500)
- Evaluated generation quality using graph-based metrics: F1 and EM at node, edge, and program levels
- Model: Llama3.1-8B-Instruct / Qwen2.5-7B-Instruct

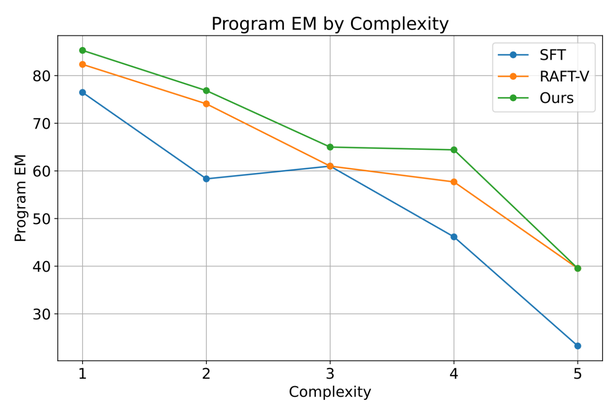
Format	Method	Node F1	Edge F1	Node EM	Edge EM	Program EM
XML	SFT	84.8 / 79.0	74.7 / 68.0	55.2 / 46.4	54.8 / 45.6	54.8 / 45.6
	RAFT-V	88.1 / **87.4**	80.9 / 78.7	63.8 / 59.6	62.8 / 58.8	62.8 / 58.8
	Ours	**89.6 / 87.4**	**82.6 / 79.0**	**66.2 / 61.2**	**65.2 / 60.0**	**65.2 / 60.0**
JSON	SFT	85.1 / 81.2	74.9 / 69.3	53.6 / 46.0	52.6 / 45.4	52.6 / 45.4
	RAFT-V	90.1 / 88.4	82.0 / 79.6	63.8 / 59.2	63.2 / 57.4	63.2 / 57.4
	Ours	**90.7 / 88.7**	**83.0 / 80.5**	**66.0 / 61.6**	**65.2 / 60.2**	**65.2 / 60.2**
Metaprogram	SFT	86.3 / 81.7	75.7 / 70.2	55.2 / 47.6	54.0 / 47.2	54.0 / 47.2
	RAFT-V	89.9 / 89.1	82.4 / 80.4	64.6 / 60.6	63.8 / 59.6	63.8 / 59.6
	Ours	**90.6 / 89.7**	**83.6 / 81.6**	**68.6 / 63.2**	**67.2 / 61.8**	**67.2 / 61.8**

Result: Our method **improves Program EM by up to +13.2% over SFT**, and the gains are consistent across **formats** and **models**.

Analysis

Methods	Node F1	Edge F1	Node EM	Edge EM	Program EM
RAFT-V (100%)	90.6	**83.6**	66.6	65.2	65.2
RAFT-V	89.9	82.4	64.6	63.8	63.8
BoN (Random)	90.4	83.2	66.8	65.6	65.6
BoN (Unstrict)	**90.7**	83.5	67.0	65.8	65.8
BoN (Strict)	90.5	83.3	67.4	66.2	66.2
Ours (Random)	90.6	83.5	68.0	66.6	66.6
Ours	90.6	**83.6**	**68.6**	**67.2**	**67.2**

Graph-editing yields **more effective preference pairs** than BoN sampling, especially when paired with GED-based selection.



2-stage training consistently outperforms, **widening the gap at higher complexities** (+18.3% in 4, +16.3% in 5)

k	Node F1	Edge F1	Node EM	Edge EM	Program EM
1 (Ours)	89.9	82.4	64.6	63.8	63.8
2	90.6	83.5	**66.4**	**65.6**	**65.6**
3	**90.9**	**84.0**	65.4	64.6	64.6

Single retrieved example is sufficient for strong performance

τ	Node F1	Edge F1	Node EM	Edge EM	Program EM
0	**90.7**	83.4	67.2	65.8	65.8
0.1 (Ours)	90.6	**83.6**	**68.6**	**67.2**	**67.2**
0.5	90.5	83.3	68.2	66.8	66.8
0.9	90.6	83.4	67.4	66.0	66.0

Small deletion ratio (0.1) in graph editing yields best performance

Contributions

1. Demonstrate that **training-based methods** outperform prompting for industrial VPL (LD) generation.
2. Propose a **2-stage approach** with retrieval and preference optimization, achieving **notable gains over SFT**.
3. Present the **first LLM-based LD generator** deployable in real-world **factory settings.**

Contact: deokhk@postech.ac.kr / jeonghuncho@postech.ac.kr